DIM0888 – Image Processing
Homework 1

Rafael Garcia

October 12, 2026

Implementation Overview

Questions 2 to 5 were implemented in C++17 and built with CMake (tested with GCC 13.3 on Ubuntu). The source code is available at `https://github.com/garciaRafa/image-processing` and is also included in the submitted archive. All processing algorithms (convolution, histogram equalization, median filtering, and the construction and application of the frequency-domain filters, including padding and centering) were implemented from scratch. Two external libraries were used, only for tasks outside the scope of the assignment:

- **FFTW3**, for the forward and inverse discrete Fourier transforms (Question 5), as allowed by the assignment;
- **stb_image** and **stb_image_write**, single-header public-domain libraries, only to decode and encode PNG files. PGM files (both the ASCII `P2` and the binary `P5` variants used by the test images) are read by a parser written for this work.

Images are loaded as single-channel arrays of floating-point values, so intermediate results, such as the negative responses of high-pass filters, are not truncated. Color inputs are converted to luminance when loaded, and values are rounded and clipped to $[0, 255]$ only when an image is saved. Signed responses (Laplacian and pure high-pass outputs) are displayed with zero mapped to mid-gray, as described in Section 2. Table 1 summarizes the organization of the code. A single executable, `pdi`, exposes each operation as a subcommand, and one script per question reproduces all the results shown in this report.

Table 1: Organization of the source code.

File	Contents	Question
`src/image.{hpp,cpp}`	Image type, PGM/PNG I/O, display scaling	–
`src/convolution.{hpp,cpp}`	Spatial convolution and kernels	2
`src/histogram.{hpp,cpp}`	Histogram and histogram equalization	3
`src/median.{hpp,cpp}`	Median filter	4
`src/hfe.{hpp,cpp}`	High-frequency emphasis, spectrum	5
`src/main.cpp`	Command-line interface (`pdi`)	–
`scripts/run_q{2,3,4,5}.sh`	Generation of all results	2–5

Building and running. On Ubuntu, with the test images in the `data/` directory:

```
sudo apt install build-essential cmake libfftw3-dev
cmake -S . -B build && cmake --build build
./scripts/run_q2.sh   # likewise run_q3.sh, run_q4.sh and run_q5.sh
```

The results are written to `results/q2` to `results/q5`. Running `./build/pdi` without arguments lists all subcommands, for example `./build/pdi median input.pgm output.png 3`.

1 Sampling and Quantization

Definitions. Digitizing a continuous signal requires discretizing both its domain and its amplitude. *Sampling* is the discretization of the domain: the signal is measured only at a finite set of positions, which, for images, determines the spatial resolution (number of pixels). *Quantization* is the discretization of the amplitude: each measured value is mapped to one of a finite number of levels, which, for images, determines the number of gray levels each pixel can assume. Sampling limits the finest spatial detail that can be represented, while quantization limits the finest intensity variation.

Digitization of $f(x)$. The function $f(x) = -0.5x^2 + 3.5x + 1$ was sampled on $[0, 5]$ with a sampling interval of 0.5 (i.e., a sampling rate of 2 samples per unit), yielding 11 samples. Since gray levels 0 and 15 correspond to $f = 0$ and $f = 7.5$, the 16 levels are uniformly spaced with quantization step

$$\Delta = \frac{7.5 - 0}{15} = 0.5, \tag{1}$$

so level k represents the value $0.5k$. Each sample was assigned to the nearest level,

$$k = \text{round}\left(\frac{f(x)}{\Delta}\right) = \text{round}\left(2f(x)\right), \tag{2}$$

and encoded with $\log_2 16 = 4$ bits. The maximum of f occurs at $x = -b/(2a) = 3.5$, where $f(3.5) = 7.125 < 7.5$, so no sample is clipped. Table 2 lists the results. Since every value of $2f(x)$ has a fractional part of either 0 or 0.25, rounding and truncation produce the same levels.

Table 2: Sampling and quantization of $f(x)$ with $\Delta = 0.5$.

x	$f(x)$	$f(x)/\Delta$	Level	Code	Error
0.0	1.000	2.00	2	`0010`	0.000
0.5	2.625	5.25	5	`0101`	0.125
1.0	4.000	8.00	8	`1000`	0.000
1.5	5.125	10.25	10	`1010`	0.125
2.0	6.000	12.00	12	`1100`	0.000
2.5	6.625	13.25	13	`1101`	0.125
3.0	7.000	14.00	14	`1110`	0.000
3.5	7.125	14.25	14	`1110`	0.125
4.0	7.000	14.00	14	`1110`	0.000
4.5	6.625	13.25	13	`1101`	0.125
5.0	6.000	12.00	12	`1100`	0.000

The resulting 44-bit digital signal (11 samples × 4 bits) is

```
0010 0101 1000 1010 1100 1101 1110 1110 1110 1101 1100
```

The maximum quantization error is 0.125, below the theoretical bound $\Delta/2 = 0.25$. Note that the samples at $x = 3.0$, 3.5 and 4.0 are mapped to the same level, so the peak of f is lost after quantization (Figure 1). This illustrates how a small number of levels flattens fine amplitude variations, the one-dimensional analog of false contouring in images.

2 Spatial Convolution

Implementation. The function `convolve(img, kernel, d)` receives the image, the kernel weights (row-major, $d \times d$) and the kernel size d, which must be odd. With $r = \lfloor d/2 \rfloor$, it

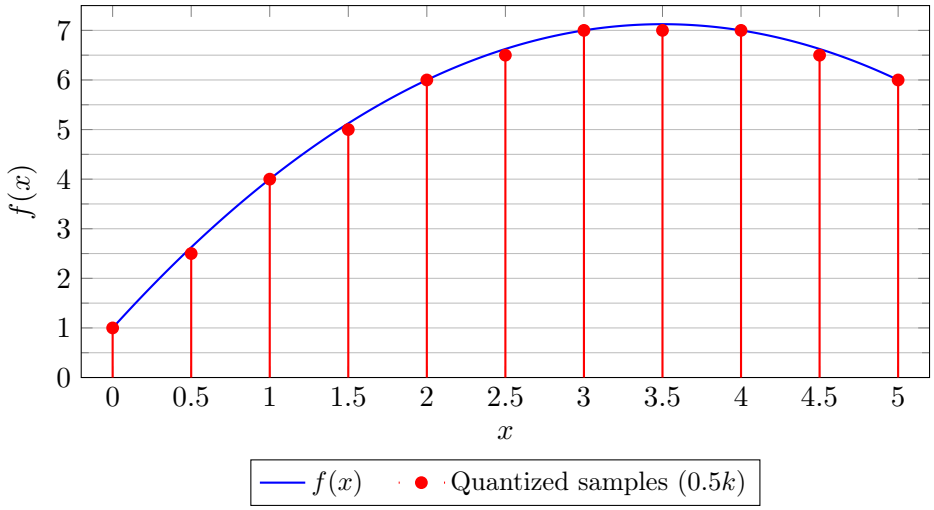


Figure 1: Continuous function $f(x)$ and its sampled and quantized version. Horizontal grid lines mark the quantization levels ($\Delta = 0.5$).

computes

$$g(x, y) = \sum_{s=-r}^{r} \sum_{t=-r}^{r} w(s, t)\, f(x - s,\, y - t), \tag{3}$$

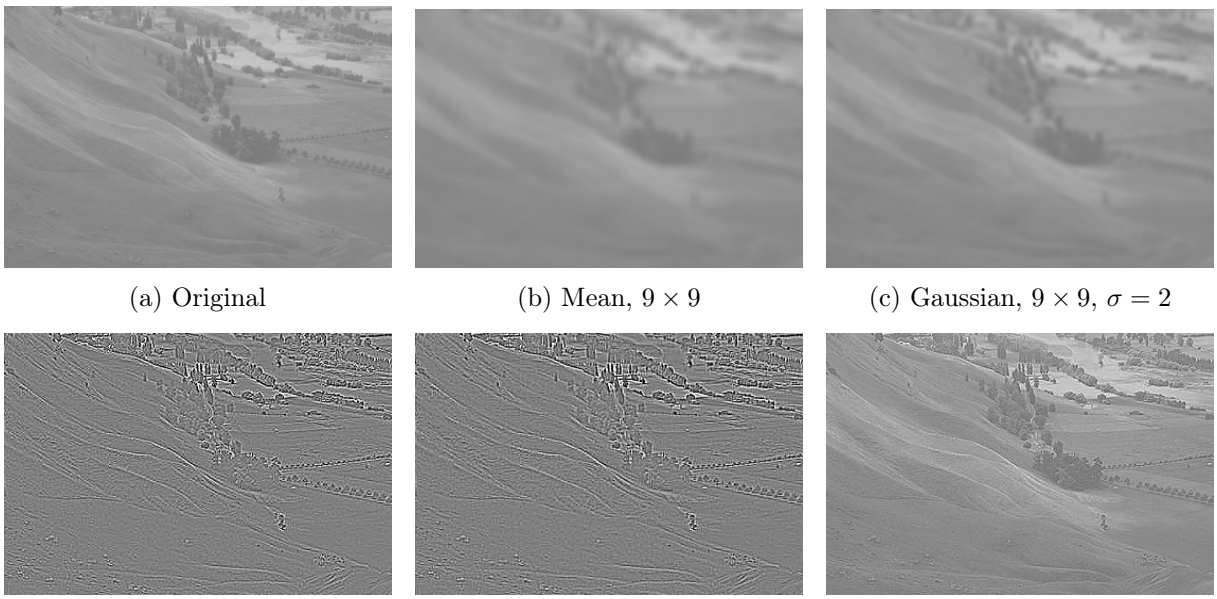
that is, a true convolution with the kernel rotated by 180° (not a correlation). As required, the first and last r rows and columns, where the kernel does not fit entirely inside the image, are ignored, so an $M \times N$ input produces an $(M - 2r) \times (N - 2r)$ output. All computations are performed in floating point. The implementation was validated by convolving a unit impulse with an asymmetric 3×3 kernel, which reproduces the kernel itself, and by checking that a constant image is preserved by the mean filter and zeroed by the Laplacian. Color inputs are converted to grayscale (luminance) when loaded.

Kernels. For low-pass filtering, the $d \times d$ mean (box) kernel, with weights $1/d^2$, and the normalized Gaussian kernel, $w(s, t) \propto e^{-(s^2+t^2)/2\sigma^2}$, were used. Both have weights that sum to one, so the mean intensity is preserved. For high-pass filtering, the 4- and 8-neighbor Laplacians were used,

$$L_4 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \qquad L_8 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}, \tag{4}$$

together with the sharpening kernel $\delta + L_4$, which adds the Laplacian response back to the image, $g = f + \nabla^2 f$ (with the sign convention of L_4). Since the Laplacian weights sum to zero, its response is signed and centered on zero. For display, zero was mapped to mid-gray (128) and the response was divided by the 99th percentile of its absolute value, with saturation beyond it. Plain min–max scaling was avoided because a few extreme responses compress the typical edge response into a narrow band around mid-gray, making edges barely visible. The other outputs were clipped to $[0, 255]$.

Discussion. The low-pass kernels attenuate fine detail and noise, and the blurring increases with the kernel support. For the same 9×9 support, the box filter blurs more strongly than the Gaussian with $\sigma = 2$: the box weights all 81 neighbors equally, whereas the Gaussian concentrates most of its weight within about 2σ of the center, so its effective support is smaller and edges are better preserved. The Gaussian also avoids the artifacts of the box filter, whose frequency response has side lobes. The Laplacian outputs are close to zero (mid-gray) in homogeneous



(a) Original (b) Mean, 9×9 (c) Gaussian, 9×9, $\sigma = 2$

(d) Laplacian L_4 (e) Laplacian L_8 (f) Sharpened, $\delta + L_4$

Figure 2: Low-pass (top) and high-pass (bottom) filtering of `Hill.png`.



(a) Original (b) Mean, 9×9 (c) Sharpened, $\delta + L_4$

Figure 3: Spatial filtering of `RX_a.png`.

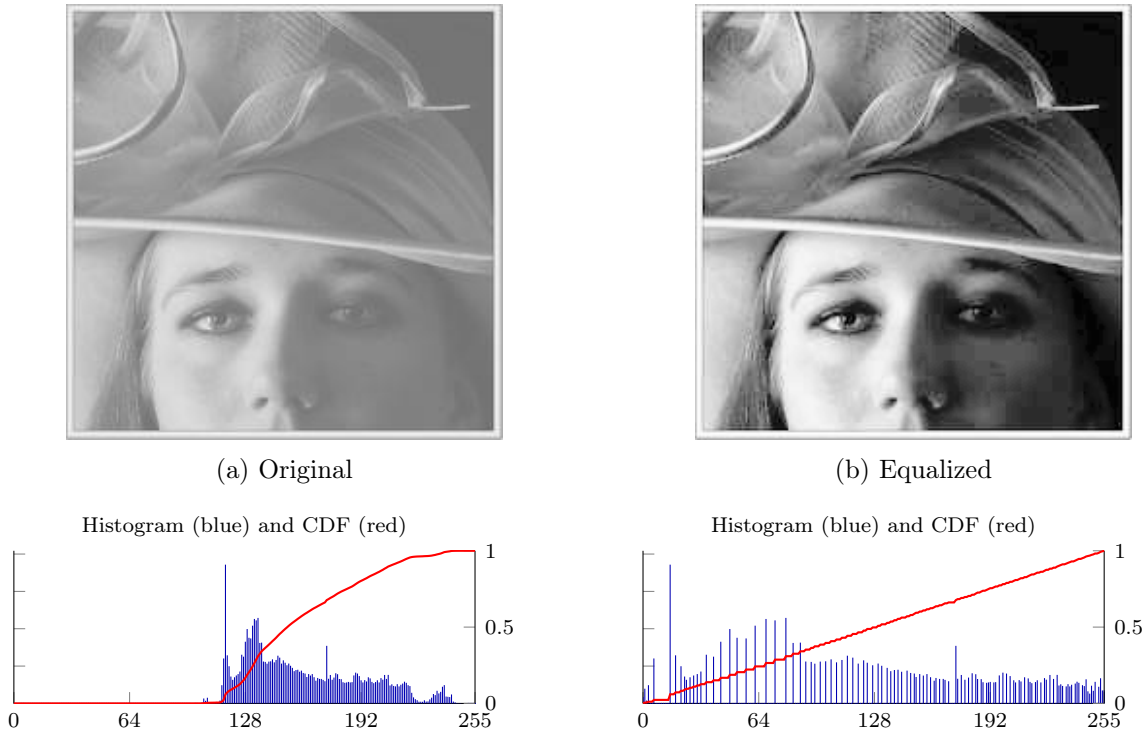
regions and respond with paired dark and bright values on either side of edges and fine structures; L_8, which also includes the diagonal neighbors, gives a stronger and more isotropic response than L_4. Adding the Laplacian back to the image produces a markedly crisper result: edges and fine texture (vegetation, terrain) are strongly accentuated, which also increases local contrast and amplifies noise, illustrating the usual trade-off of high-pass sharpening.

3 Histogram Equalization

Implementation. Let n_k be the number of pixels with gray level r_k, $k = 0, \ldots, L-1$ ($L = 256$), in an $M \times N$ image, so that $p_r(r_k) = n_k/MN$ is the normalized histogram. Histogram equalization maps each level through the scaled cumulative distribution function (CDF) of the input,

$$s_k = T(r_k) = \text{round}\left((L-1) \sum_{j=0}^{k} p_r(r_j) \right), \qquad k = 0, \ldots, L-1. \tag{5}$$

The implementation computes the histogram in a single pass over the image, builds the look-up table T from the running sum of p_r, and applies it to every pixel, at a cost of $O(MN + L)$. In the continuous case, T yields a uniform output density; in the discrete case, levels cannot be split, so the output histogram is only approximately flat, but its CDF becomes approximately linear. As a check, an image with 20 levels uniformly distributed in $[100, 119]$ was mapped to 20 levels evenly spread over $[14, 255]$, with output CDF deviating at most 0.002 from a straight line. Since T is monotonic, the ordering of gray levels is preserved, and the number of distinct levels can only stay the same or decrease (levels with small p_r may merge after rounding).



(a) Original (b) Equalized

Figure 4: Histogram equalization of `woman_low_contrast.pgm`.

Discussion. The original images occupy a narrow band of gray levels (for instance, the `woman` image only spans $[98, 250]$), so their histograms are concentrated and their CDFs rise steeply over a short interval. After equalization, the occupied levels are spread over the full $[0, 255]$ range and

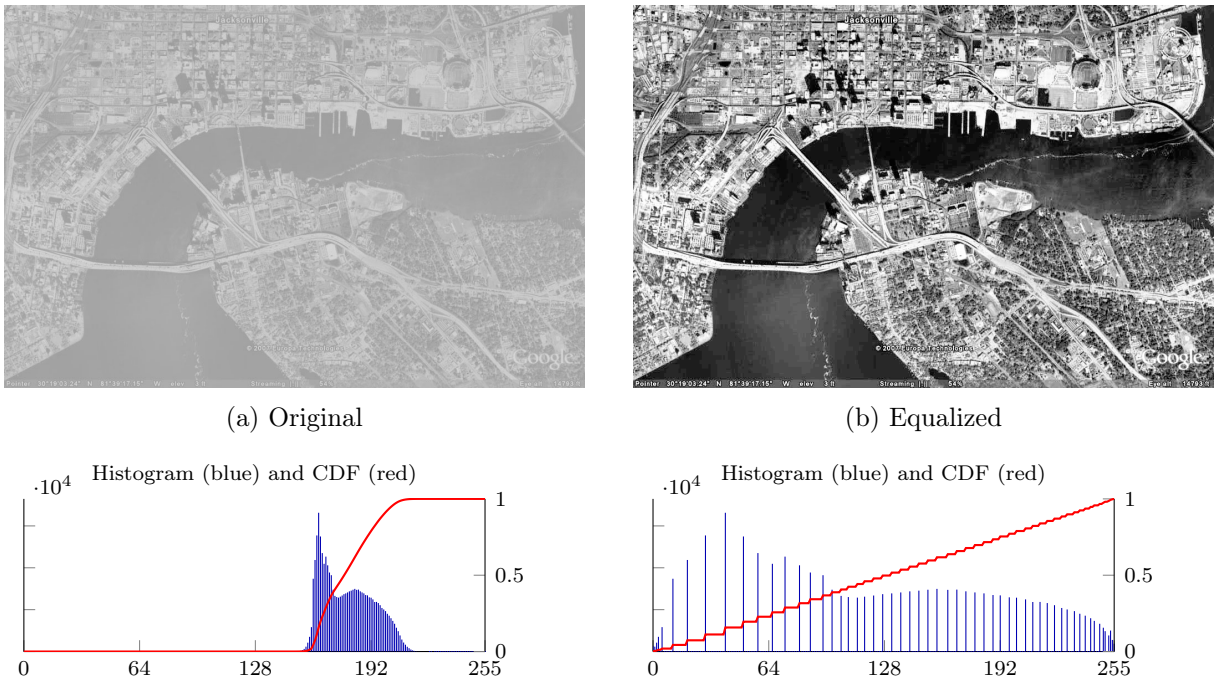


Figure 5: Histogram equalization of `Google_JAX_low_contrast.pgm`.

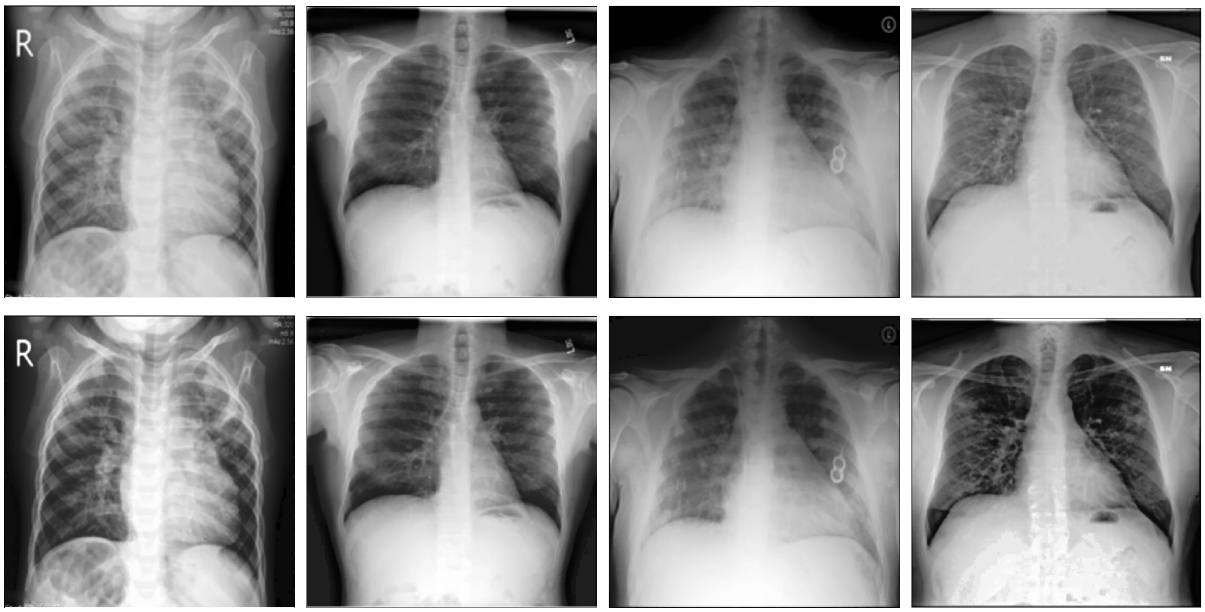


Figure 6: Radiographs `RX_a` to `RX_d`: originals (top) and equalized (bottom).

the CDF becomes close to a straight line, which increases global contrast and makes details in the previously compressed tonal range visible. The output histograms are not perfectly flat: they show gaps between occupied levels, because equalization redistributes, but cannot create, gray levels. Since the transformation is driven by the most frequent levels, large homogeneous regions (such as backgrounds) receive a large share of the output range, which can over-enhance them and amplify noise and compression artifacts; this is a known limitation of global equalization, addressed by local or contrast-limited variants (CLAHE).

4 Median Filter

Implementation. The median filter replaces each pixel by the median of the gray levels in its $d \times d$ neighborhood (d odd),

$$g(x, y) = \text{median}\{\, f(x + s,\, y + t) : |s|, |t| \leq \lfloor d/2 \rfloor \,\}. \tag{6}$$

For each pixel, the d^2 values of the window are copied to a buffer and the median is found with a partial selection (`std::nth_element`), which runs in $O(d^2)$ on average instead of the $O(d^2 \log d^2)$ of a full sort. Unlike the convolution of Section 2, the borders were not discarded: the image is extended by replicating its edge pixels, so the output keeps the input size and impulses near the borders are also removed. Since the filter is non-linear, it cannot be written as a convolution. As a quantitative measure, the percentage of pixels equal to 0 or 255, the two values produced by salt-and-pepper noise, was computed before and after filtering. This slightly overestimates the remaining noise, because genuine black or white pixels of the scene are also counted.

Table 3: Percentage of pixels equal to 0 or 255 before and after median filtering.

Image	Median filter	Before (%)	After (%)
bear	3×3	25.02	0.50
boat	3×3	25.00	0.55
glass	3×3	5.32	0.04
lungs (10%)	3×3	9.94	0.12
lungs (10%)	5×5	9.94	0.06
lungs (10%)	7×7	9.94	0.02
lungs (10%)	3×3 (3 passes)	9.94	0.10
lungs (30%)	3×3	29.40	1.14
lungs (30%)	5×5	29.40	0.07
lungs (30%)	7×7	29.40	0.04
lungs (30%)	3×3 (3 passes)	29.40	0.11
lungs (50%)	3×3	48.71	8.96
lungs (50%)	5×5	48.71	0.62
lungs (50%)	7×7	48.71	0.05
lungs (50%)	3×3 (3 passes)	48.71	0.95

Discussion. Salt-and-pepper noise consists of isolated pixels set to the extremes of the gray scale. Because these values are outliers within each window, the median ignores them as long as they are fewer than half of the d^2 samples, and the impulses are removed while edges are preserved, since the output is always one of the original neighboring values. The linear mean filter (Figure 9), in contrast, spreads each impulse over its neighborhood, producing gray blotches and blurring edges. The 3×3 median is sufficient for moderate noise, but it breaks down as the noise density increases: at 50%, many 3×3 windows contain five or more impulses, so the median

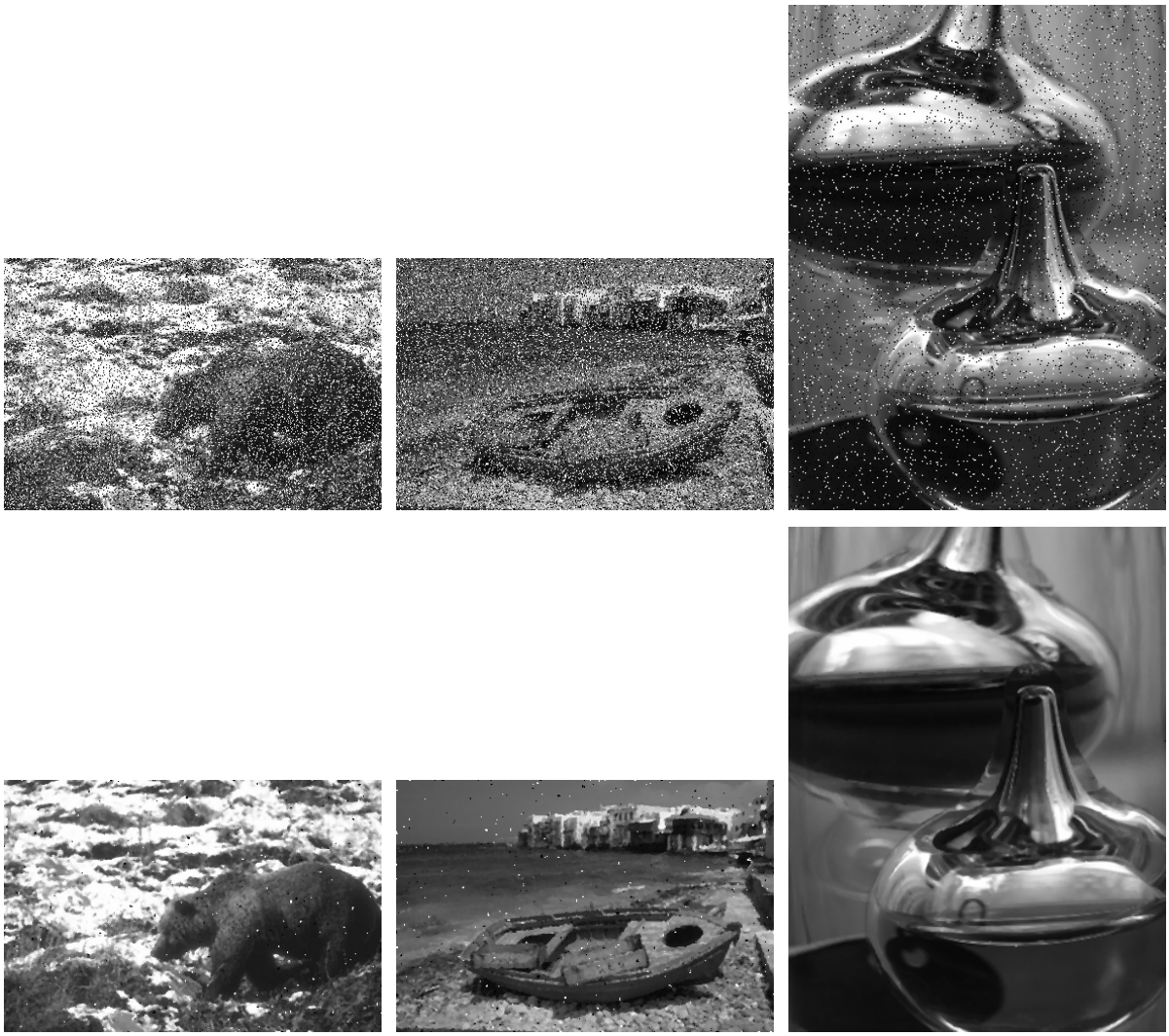


Figure 7: Salt-and-pepper images (top) and the result of a 3×3 median filter (bottom).

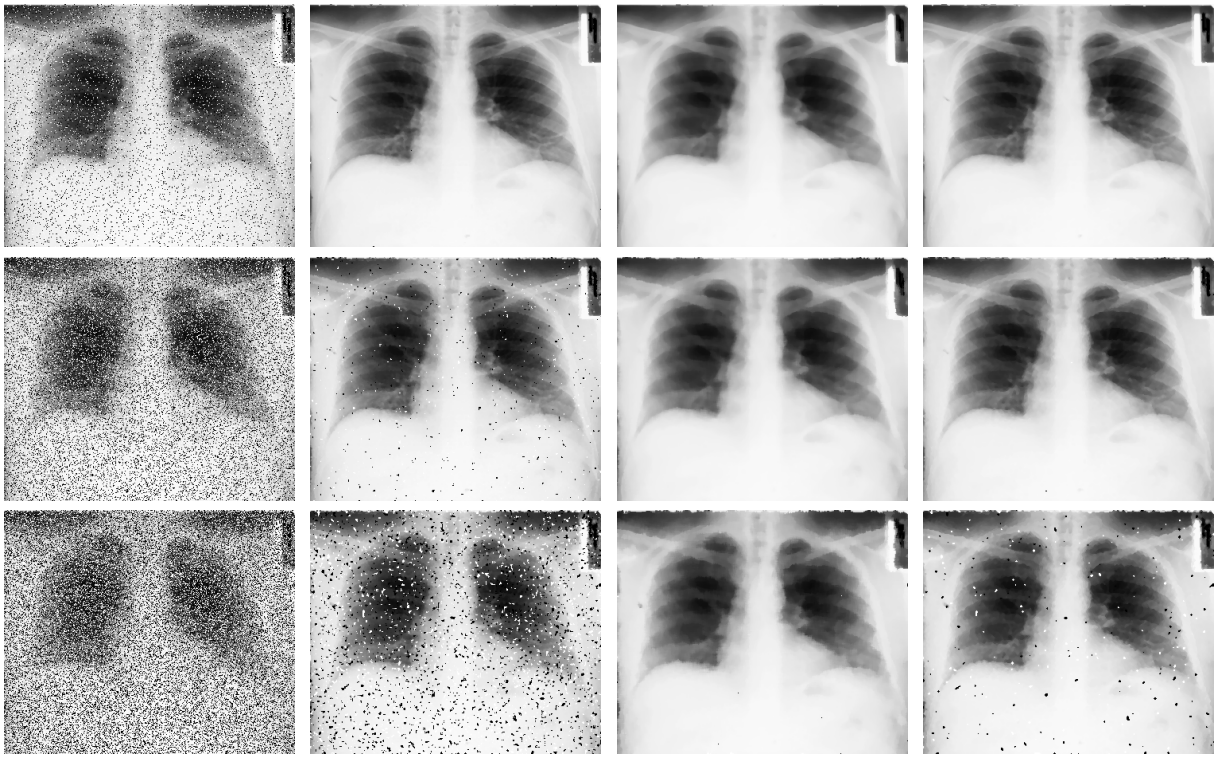
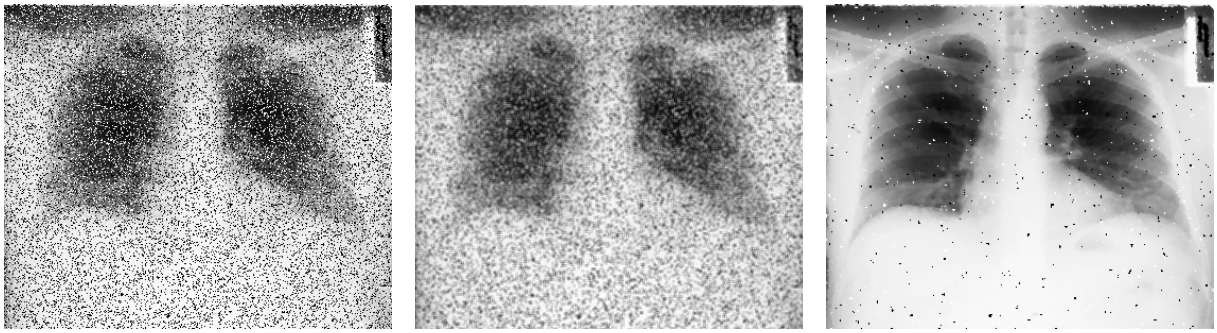


Figure 8: Lung image with 10%, 30% and 50% salt-and-pepper noise (rows). Columns: noisy input, 3×3 median, 7×7 median, and three passes of the 3×3 median.



(a) 30% noise (b) Mean, 3×3 (c) Median, 3×3

Figure 9: Linear (mean) versus non-linear (median) filtering with the same 3×3 support.

itself is an impulse and residual noise remains (Table 3). Larger windows or repeated passes remove the residual impulses, at the cost of more smoothing of fine detail, which illustrates the trade-off between noise removal and detail preservation; adaptive median filters, which enlarge the window only where needed, address this limitation.

5 High-Frequency Emphasis Filter

Filter definition. High-frequency emphasis filtering computes

$$g(x,y) = \mathcal{F}^{-1}\big\{[k_1 + k_2 H_{HP}(u,v)]\, F(u,v)\big\}, \tag{7}$$

where $k_1 \geq 0$ controls how much of the original spectrum is kept (in particular the DC term, since $H_{HP}(0,0) = 0$, so the mean intensity of g is k_1 times the mean of f) and $k_2 > 0$ controls the gain applied to the high frequencies. A Gaussian high-pass filter was used,

$$H_{HP}(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}, \tag{8}$$

where $D(u,v)$ is the distance from (u,v) to the center of the frequency rectangle and $D_0 = 40$ is the cutoff. Unlike the ideal filter, the Gaussian has no discontinuity in its transfer function and therefore produces no ringing; a Butterworth filter of order $n = 2$, $H_{HP} = 1/[1 + (D_0/D)^{2n}]$, was also implemented for comparison.

Implementation. The DFT was computed with the FFTW3 library, following the standard frequency-domain filtering procedure:

1. The $M \times N$ image (454×291) is padded to $P \times Q = 2M \times 2N$, which avoids the wraparound error caused by the implied periodicity of the DFT. The padded region replicates the border pixels instead of using zeros, to avoid an artificial intensity step at the image boundary, which the high-pass filter would emphasize. Padding also makes both dimensions even, which is required for the centering below, since the original width $N = 291$ is odd.

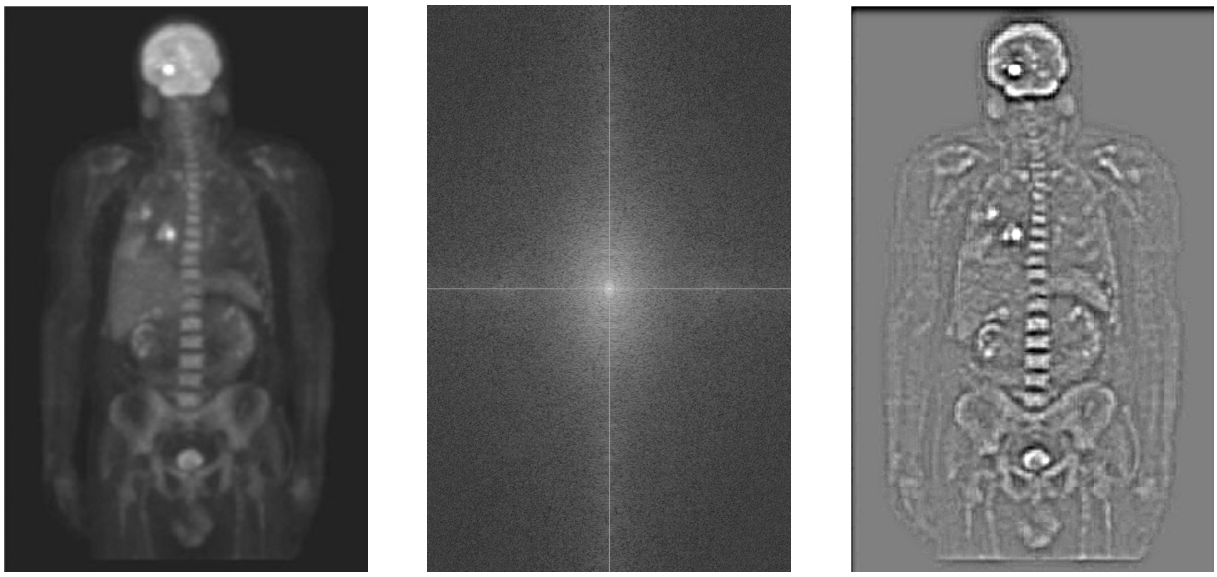
2. The padded image is multiplied by $(-1)^{x+y}$, which, by the property shown in Section 6, moves the origin of the spectrum to $(P/2, Q/2)$, so that $D(u,v)$ is measured from the center.

3. The forward DFT is computed and multiplied by $H(u,v) = k_1 + k_2 H_{HP}(u,v)$, which is real and symmetric, so the filter does not change the phase.

4. The inverse DFT is computed and divided by PQ (FFTW transforms are unnormalized); the real part is multiplied by $(-1)^{x+y}$ again and the top-left $M \times N$ region is extracted. Values outside $[0, 255]$ are clipped for display.

The implementation was validated with three tests: with $k_1 = 1$ and $k_2 \to 0$ the output is identical to the input; a constant image of value 100 filtered with $k_1 = 0.5$ becomes a constant image of value 50, confirming that only k_1 acts on the DC term; and the spectrum of a constant image has its peak exactly at $(P/2, Q/2)$.

Discussion and choice of parameters. The parameters were chosen according to four criteria, evaluated on the same anatomical structures in all results: (i) sharpness of fine detail, such as the separation between vertebrae and the focal uptake in the right thorax; (ii) preservation of the relative intensities between regions (brain, liver, background), which carry the diagnostic information in PET, since gray level reflects tracer uptake; (iii) amplification of noise in the background and inside homogeneous organs; and (iv) halo artifacts around high-uptake structures, such as the brain and the bladder, caused by the overshoot of the high-pass component (Figure 10c).



(a) Original (b) $\log(1 + |F(u, v)|)$, padded (c) High-pass only ($k_1 = 0$, zero at mid-gray)

Figure 10: PET image, its centered spectrum and its high-frequency component.

Figure 11 shows that k_1 mainly controls brightness and tonal content: since $H_{HP}(0, 0) = 0$, the mean intensity of the output is k_1 times that of the input, and for $k_1 < 1$ part of the result becomes negative and is clipped. With $k_1 = 0.25$ the image is too dark and the uptake information is largely lost, leaving mostly contours. The structure of the enhancement, in turn, depends on the ratio k_2/k_1: multiplying $H(u, v) = k_1 + k_2 H_{HP}(u, v)$ by a constant only scales the output. This is visible along the diagonal of the grid, where the three results with $k_2/k_1 = 2$ show the same enhanced structures and differ only in brightness. Within the row $k_1 = 1$, increasing k_2 progressively sharpens the vertebrae and the contours of the kidneys and pelvis, but with $k_2 = 2$ dark halos appear around the head and the body outline, and the interior of the trunk becomes textured as noise is amplified. These observations are supported by the output ranges. With $k_1 = 1$, increasing k_2 from 0.5 to 1 barely changes the fraction of clipped pixels (0.26% to 0.27%), whereas $k_2 = 2$ raises it to 1.09%. The minimum of the output, which measures the undershoot responsible for the dark halos, goes from -17 with $k_2 = 1$ to -56 with $k_2 = 2$. Moreover, the clipping above 255 occurs mainly in the brain and bladder, which are already saturated in the original image (its range is $[0, 255]$), so little information is lost there.

The values $k_1 = 1.0$ and $k_2 = 1.0$ were therefore selected as the best compromise: they preserve the brightness and relative uptake of the original image while clearly improving the definition of anatomical boundaries and focal uptake, without visible halos or noticeable noise amplification. High-frequency emphasis is often followed by histogram equalization (Section 3) to increase global contrast (Figure 12c), at the cost of altering the relation between gray level and uptake. The Butterworth filter of order 2 gives a result very similar to the Gaussian, with a slightly sharper transition between the attenuated and the emphasized bands.

6 DFT Translation Property

Let $f(x, y)$ be an $M \times N$ image, with $x = 0, \ldots, M - 1$ and $y = 0, \ldots, N - 1$, and let its 2-D DFT be

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}. \tag{9}$$

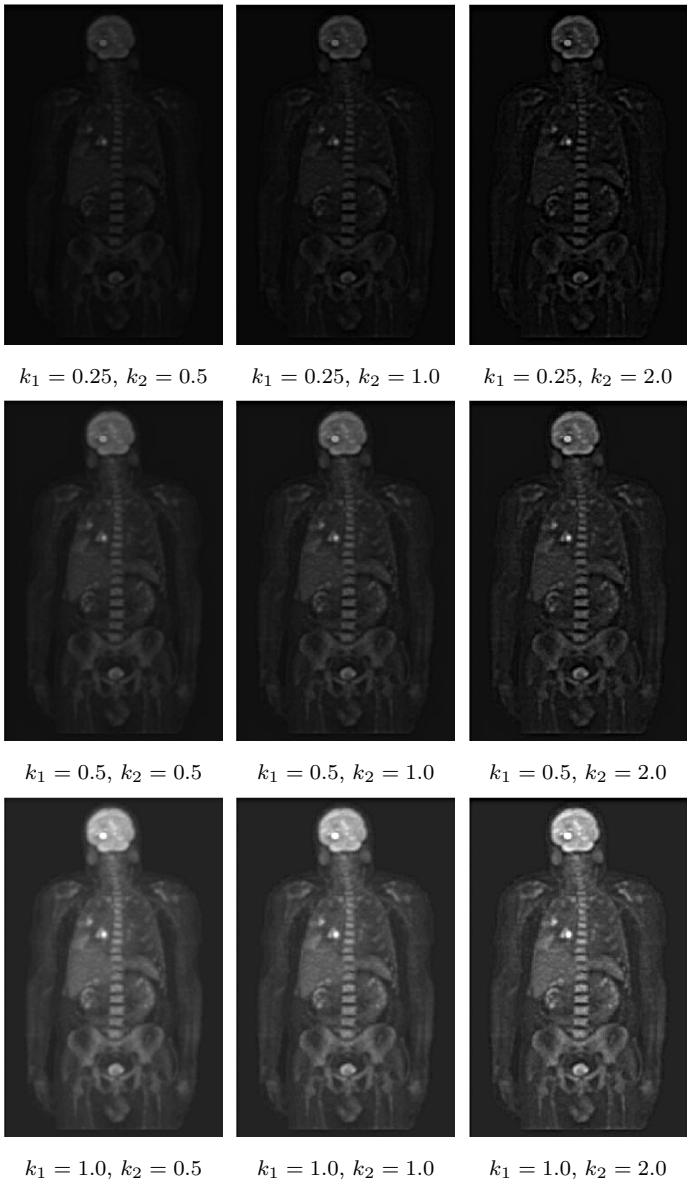
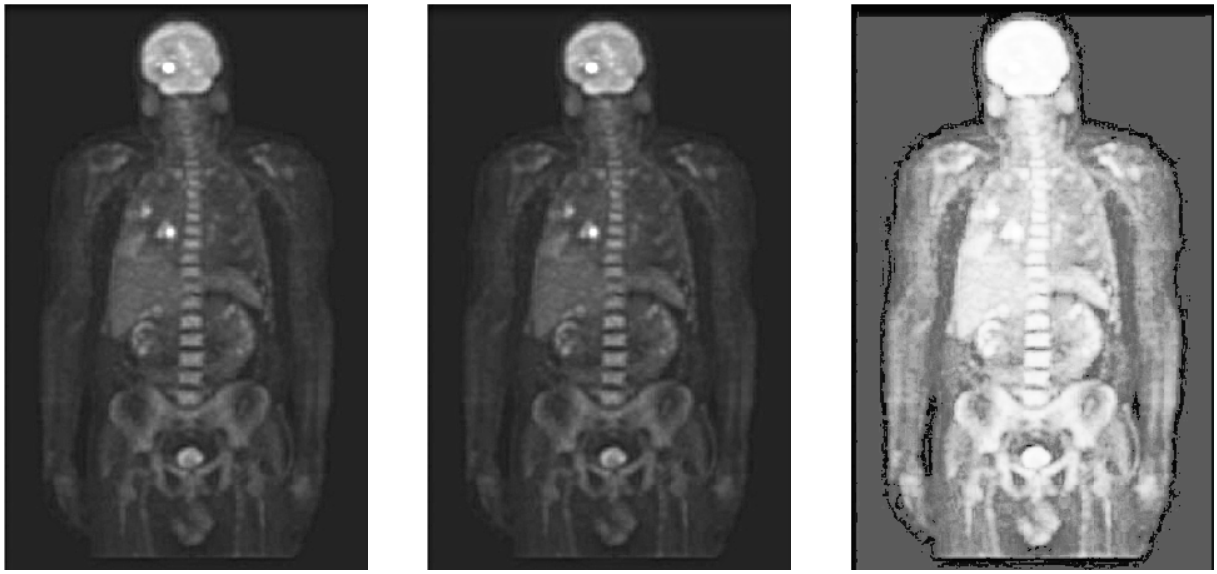


Figure 11: High-frequency emphasis with a Gaussian high-pass ($D_0 = 40$): k_1 increases from top to bottom and k_2 from left to right.



(a) Gaussian (b) Butterworth, $n = 2$ (c) Followed by equalization

Figure 12: Selected parameters ($k_1 = 1.0$, $k_2 = 1.0$): Gaussian high-pass, Butterworth high-pass, and Gaussian followed by histogram equalization.

Throughout, M and N are assumed to be even, so that $M/2$ and $N/2$ are integer frequency indices (the usual case for images, and a requirement for the shift to land on the discrete frequency grid).

Translation property. For completeness, the first property follows directly from Eq. (9). Multiplying f by the complex exponential and taking the DFT,

$$\begin{aligned}\mathcal{F}\left\{f(x,y)\, e^{j2\pi\left(\frac{u_0 x}{M}+\frac{v_0 y}{N}\right)}\right\} &= \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)\, e^{j2\pi\left(\frac{u_0 x}{M}+\frac{v_0 y}{N}\right)}\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)} \\ &= \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{(u-u_0)x}{M}+\frac{(v-v_0)y}{N}\right)} = F(u-u_0,\, v-v_0), \quad (10)\end{aligned}$$

which is Eq. (9) evaluated at $(u-u_0, v-v_0)$.

Proof of $f(x,y)(-1)^{x+y} \Leftrightarrow F(u-M/2,\, v-N/2)$. Choose $u_0 = M/2$ and $v_0 = N/2$ in Eq. (10). The exponential factor becomes

$$e^{j2\pi\left(\frac{(M/2)x}{M}+\frac{(N/2)y}{N}\right)} = e^{j\pi(x+y)} = \left(e^{j\pi}\right)^{x+y}. \quad (11)$$

By Euler's formula, $e^{j\pi} = \cos\pi + j\sin\pi = -1$, and since x and y are integers, $x+y$ is an integer, so

$$e^{j\pi(x+y)} = (-1)^{x+y}. \quad (12)$$

Substituting into Eq. (10) yields

$$\boxed{f(x,y)\,(-1)^{x+y} \;\Leftrightarrow\; F\left(u-\tfrac{M}{2},\, v-\tfrac{N}{2}\right)} \quad (13)$$

■

Interpretation. The DFT is periodic, $F(u,v) = F(u+M,v) = F(u,v+N)$, and the raw transform places the DC term $F(0,0)$ at the corner of the $[0,M-1]\times[0,N-1]$ interval, with the low frequencies split across the four corners. Multiplying the image by $(-1)^{x+y}$ before transforming moves the origin of the spectrum to $(M/2,N/2)$, so one full centered period is displayed, with low frequencies in the middle and high frequencies toward the borders. This is convenient both for visualizing $|F(u,v)|$ and for defining radially symmetric filters such as $H_{HP}(u,v)$ in Section 5, whose distance $D(u,v)$ is then measured from the center. Since $(-1)^{x+y}$ is real and equal to ± 1, the operation is trivially undone by multiplying the result of the inverse transform by $(-1)^{x+y}$ again. Analogously, setting $x_0 = M/2$ and $y_0 = N/2$ in the second property gives $f(x-M/2,\, y-N/2) \Leftrightarrow F(u,v)\,(-1)^{u+v}$, which shifts the image instead of its spectrum.

7 Restoration of Blurred Coin Photographs

Problem model. The photographs are degraded by defocus, which can be modeled as a linear, position-invariant degradation followed by additive noise,

$$g(x,y) = h(x,y) \star f(x,y) + \eta(x,y) \quad \Longleftrightarrow \quad G(u,v) = H(u,v)\,F(u,v) + N(u,v), \tag{14}$$

where f is the sharp image of the coin, h is the point spread function (PSF) of the out-of-focus camera, $\star$ denotes convolution and η is noise. Restoration amounts to estimating f from g, which requires knowing h. For a defocused lens, the PSF is approximately a uniform disk (the circle of confusion) of radius R,

$$h(x,y) = \begin{cases} 1/(\pi R^2), & x^2+y^2 \le R^2, \\ 0, & \text{otherwise}, \end{cases} \qquad H(u,v) = \frac{2J_1(2\pi R\rho)}{2\pi R\rho}, \quad \rho = \sqrt{u^2+v^2}, \tag{15}$$

where J_1 is the Bessel function of the first kind. H has circular rings of zeros, so the naive inverse filter $\hat{F} = G/H$ is not usable: it divides by values close to zero and amplifies noise without bound. The key fact of the problem is that the camera and coins of the same era are still available, which allows h to be *measured* instead of guessed. Figure 13 summarizes the proposed procedure.

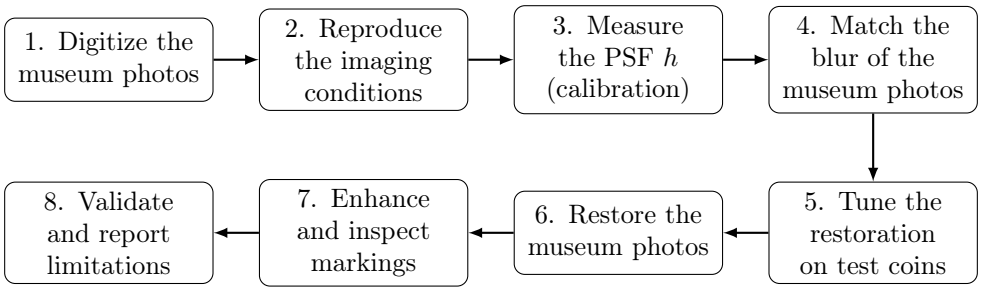


Figure 13: Proposed procedure for restoring the photographs of the stolen coins.

Proposed solution.

1. **Digitize the museum photographs** at the highest available quality: the original digital files, if they exist, or high-resolution scans of the prints or negatives. The images are converted to grayscale and kept in floating point; if the camera applies a nonlinear (gamma) response, it is inverted, since Eq. (14) only holds for intensities proportional to light.

2. **Reproduce the imaging conditions.** Using the same camera and lens, the representative coins are photographed with the same lighting, background, distance and lens settings

(focal length, aperture) used by the museum, as far as they can be inferred from the photographs, their metadata and the museum's standard procedure. Coins of the same era have similar size, relief and material, so they reproduce the image formation conditions of the stolen coins.

3. **Measure the PSF by experimentation.** With the camera deliberately defocused, two calibration images are acquired: (a) an image of an approximate impulse, such as a small bright point (a pinhole illuminated from behind) or a small reflective bead; since the image of an impulse of intensity A is $A\,h$, the PSF is obtained directly, or, equivalently, $H(u,v) = G(u,v)/A$; and (b) a sharp (in focus) and a blurred image of the same representative coin, from which $H \approx G/F$ is estimated in the frequencies where $|F|$ is not small. Because the exact defocus of the museum photographs is unknown, this calibration is repeated for a series of focus settings, producing a family of PSFs, $h_1, h_2, \ldots$, of increasing radius. Fitting the disk model (possibly convolved with a small Gaussian, to account for lens aberrations) to each measured PSF reduces the effect of noise in the measurements.

4. **Match the blur of the museum photographs.** The PSF that best explains the museum photographs is selected from the calibration family. Two cues are available in the photographs themselves: the width of the transition across the rim of the coin (the edge spread function, whose width is the diameter of the blur disk) and the radii of the dark rings in the log-magnitude spectrum $\log(1 + |G(u,v)|)$, which coincide with the zeros of H and therefore determine R. This step also checks the assumption that the blur is uniform over the coin, which holds if the coin was flat and parallel to the image plane.

5. **Tune the restoration on test coins.** The inverse filter is replaced by a regularized filter, the Wiener filter,
$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2 + K}\right] G(u,v), \tag{16}$$
where K approximates the noise-to-signal power ratio, or, alternatively, the constrained least squares filter, which replaces K by $\gamma|P(u,v)|^2$, with P the transform of the Laplacian. Since for the representative coins both the blurred and the sharp images are available, K (or γ) is chosen objectively, by minimizing the mean squared error between the restored and the sharp images, and visually, by checking the legibility of their dates and markings. As in Section 5, images are padded before the DFT, and their borders are tapered to reduce ringing caused by the boundary discontinuities.

6. **Restore the museum photographs** with the selected PSF and regularization parameter. Since legibility is the goal, results for a small range of values of K and of R around the selected ones are produced and compared, because a slight error in R is a common source of ringing artifacts.

7. **Enhance and inspect the markings.** Once restored, the images can be enhanced with the techniques of this homework, such as histogram equalization (Section 3) or high-frequency emphasis (Section 5), to improve the contrast of the relief, and examined at full resolution, ideally by the professor, who knows which inscriptions to expect.

8. **Validate and report the limitations.** The same pipeline is applied to blurred photographs of representative coins with known markings, which shows how much detail can be reliably recovered. The frequencies at the zeros of H, and those where the noise dominates, are lost and cannot be recovered by any linear filter; therefore, if the blur is severe, some fine markings may remain illegible, and the restored images should be presented together with this assessment.

8 Final Project Proposal

The final project will apply image processing to seismic data, treating common midpoint (CMP) gathers as two-dimensional images (offset $\times$ time). Besides the primary reflections, these gathers contain random noise and coherent events, such as ground roll and multiple reflections, that degrade later processing steps. The project will implement and compare noise and multiple attenuation techniques for CMP gathers, following Yilmaz (*Seismic Data Analysis*, SEG, 2001).